```
# Non-Recursive Fibonacci Program
n = int(input("Enter the number of terms: "))
a = 0
b = 1
if n <= 0:
   print("Please enter a positive number")
else:
   print("Fibonacci Series:")

   for i in range(n):
      print(a, end=" ")

      c = a + b
      a = b
      b = c

# Recursive Fibonacci Program
def fibonacci(n):

   if n == 0:
      return 0

   elif n == 1:
      return 1

   else:
      return fibonacci(n-1) + fibonacci(n-2)

n = int(input("Enter the number of terms: "))

if n <= 0:
```

```
  print("Please enter a positive number")
else:
  print("Fibonacci Series:")

  for i in range(n):
    print(fibonacci(i), end=" ")
```

theory: Recursion is a programming technique in which a function calls itself to solve a problem. The Fibonacci series is a sequence of numbers in which each number is the sum of the previous two numbers.

$$F(n) = F(n-1) + F(n-2)$$

A non-recursive approach is a method in which a function does not call itself.It uses loops such as for or while to solve a problem.

A recursive approach is a method in which a function calls itself to solve a problem.A recursive function has two important parts:

1. Base condition: The condition that stops the recursive calls.2.Recursive call: The function calls itself with a smaller input.

| 2. Parameter | Non-recursive | Recursive |
|---|---|---|
| Technique | Looping | Function calling itself |
| Time complexity | O(n) | $O(2^n)$ |
| Auxiliary space | O(1) | O(n) |
| Execution | Faster | Slower |
| Memory usage | Less | More |
| Base condition | Not required | Required |

Conclusion: The non-recursive approach is more efficient than the simple recursive approach because it takes O(n) time and O(1) auxiliary space, whereas the recursive approach takes $O(2^n)$ time and O(n) auxiliary space.

What is a function call stack?

Ans: It is a memory structure that stores information about active function calls, including their local variables and return addresses.

**Tricky external questions**

Q24. What is the difference between recursion and iteration?

Ans: Recursion uses function calls to repeat operations, whereas iteration uses loops such as for and while.

Q25. Is recursion always slower than iteration?Ans: No, but in this Fibonacci example, simple recursion is slower because it recalculates the same values repeatedly

.Q26. What is the disadvantage of recursion?Ans: It consumes additional stack memory and may cause a recursion depth error for large inputs.

Q27. Can Fibonacci be calculated without recursion and loops?Ans: Yes, using mathematical formulas such as Binet's formula or matrix exponentiation.

Q28. What is dynamic programming?Ans: Dynamic programming is a technique that stores previously calculated results to avoid repeated calculations.

Q29. How can we improve recursive Fibonacci?Ans: By using memoization, which stores previously calculated Fibonacci values.

Q30. What happens when the user enters 1?Ans: Both programs display only 0 because we are generating one term.

The non-recursive (iterative) Fibonacci approach is better in terms of time complexity.

| **Approach** | **Time complexity** | **Efficiency** |
|---|---|---|
| Non-recursive | O(n) | Better |
| Recursive | $O(2^n)$ | Slower |

Why is non-recursive better?Non-recursive calculates each Fibonacci number only once.,Recursive calculates the same Fibonacci numbers repeatedly,Therefore, non-recursive takes less execution time, especially for large input